LoRA: Low-Rank Adaptation of Large Language Models

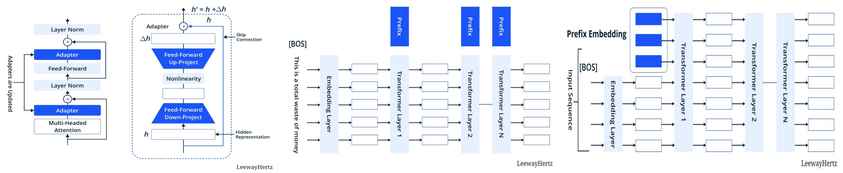
Lirong Yao · Laurence Yang · Haoyu Yan · Yaxi Zeng
CS 4782 Deep Learning

Background & Motivation

Large language models (LLMs) are expensive to fine-tune:
- Full fine-tuning requires updating all parameters
- Memory & compute cost scale with model size
- Deployment requires many task-specific model copies

Goal: Efficient adaptation with minimal parameters and compute

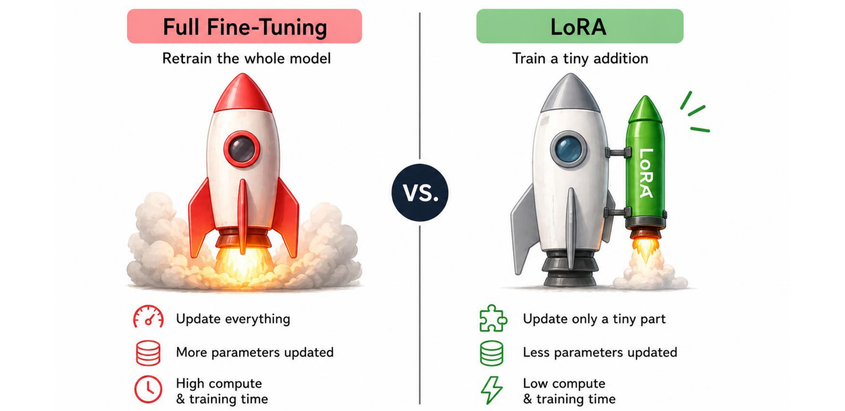
Existing approaches and their drawbacks:
- Adapters: extra latency due to added sequential layers
- Prompt / Prefix tuning: consumes context length, poor scaling, unpredictable behavior



Adapter | Prefix Tuning | Prompt Tuning

Why Train the Whole Model When You Don't Have To?

LoRA: Just patch your model!



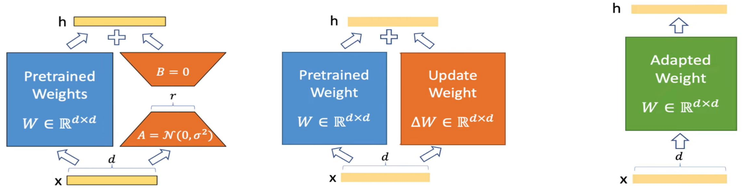
The Paper: How Does LoRA Work?

Instead of updating pretrained weights $W_0 \in \mathbb{R}^{d\times d}$, LoRA:
- Freezes W_0 (no gradient updates to original weights)
- Learns a low-rank decomposition $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times d}$, $r \ll d$

The adapted forward pass becomes:

$$W = W_0 + \Delta W \qquad \Delta W = BA$$

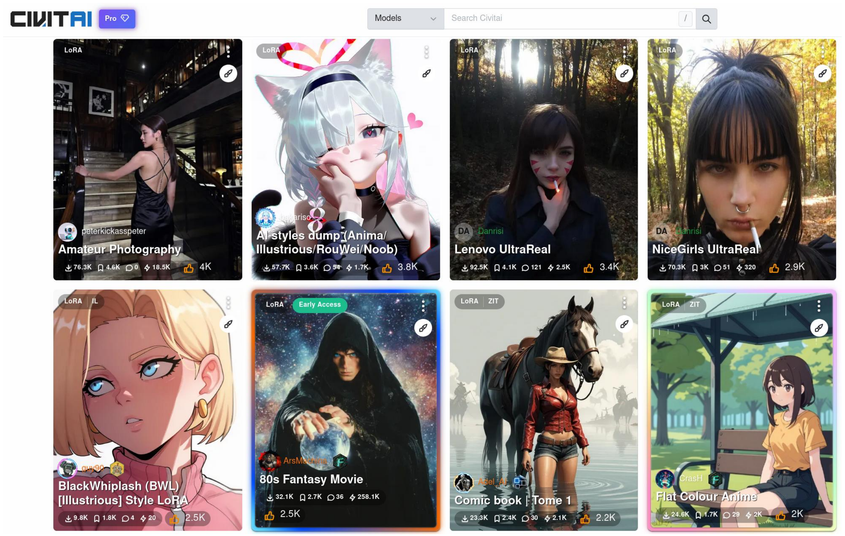
At inference: merge ΔW into W_0 — zero added latency, same architecture.
Note: This is a special case of full fine-tuning without constraints on ΔW composition.



(a) LoRA Training | (b) Full Fine-tuning | (c) LoRA Inference

LoRA introduces minimal inference latency

LoRA Beyond LLMs: Stable Diffusion



Our Experiments: WHY and WHERE? - Methodology & Results

WHY? Modularity + Efficiency! LoRA vs Not LoRA

Encoder Model (RoBERTa base) — Natural Language Understanding Tasks

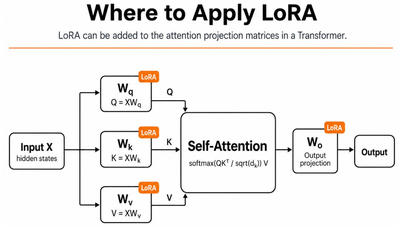
Model & Method	# Params	SST-2	MRPC
RoBERTa (Full FT)	125.0M	94.8	**90.2**
RoBERTa (AdapterDrop)	0.9M	94.7±0.3	88.4±0.1
RoBERTa (LoRA paper)	**0.3M**	**95.1±0.2**	89.7±0.7
RoBERTa (our LoRA)	**0.3M**	94.4	88.3

LoRA matches or exceeds full fine-tuning with 1000× fewer trainable parameters.
*** We also ran the experiment on GPT-2 and BLUE (language generation tasks) and observed similar patterns. Our observation matches the observations presented in the paper.**

We designed a RoBERTa-base version of the paper's LoRA target-module ablation using SST-2 as the downstream task. To keep the LoRA parameter budget approximately matched, we used r=8 when adapting one attention projection, r=4 when adapting two projections, and r=2 when adapting all four projections.

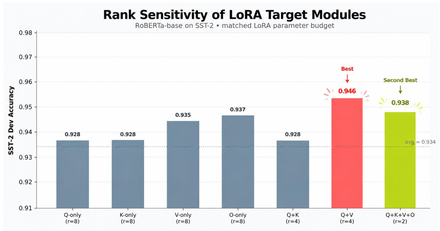
Where to Apply LoRA

LoRA can be added to the attention projection matrices in a Transformer.



Rank Sensitivity of LoRA Target Modules

RoBERTa-base on SST-2 • matched LoRA parameter budget



Whether the paper's GPT-3 finding that Wq + Wv is best also appears in a smaller encoder model like RoBERTa? -- Yes! Wq + Wv combination still appears to be the best option given our results

Summary - Advantages of LoRA

Parameter Efficiency
- Reduces trainable parameters by 10,000× (354.92M → 0.35M in GPT-2)

Exceptional Performance
- Matches or exceeds full fine-tuning across diverse tasks

Zero Inference Latency
- ΔW merged into W_0 at deployment — identical architecture & speed

Modularity
- Swap LoRA modules per task without reloading the base model

References

- Original paper: *LoRA: Low-Rank Adaptation of Large Language Models* by Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, https://arxiv.org/abs/2106.09685
- PEFT architecture images: *Parameter-efficient Fine-tuning (PEFT): Overview, benefits, techniques and model training* by LeewayHertz, https://www.leewayhertz.com/parameter-efficient-fine-tuning/
- LoRA architecture images: *What is Low-Rank Adaptation (LoRA) | explained by the inventor* by Edward Hu, https://www.youtube.com/watch?v=DhRoTONcyZE
- Examples for LoRA in diffusion model: Civitai, https://civitai.com